

TransMASK: Masked State Representation through Learned Transformation

Author Names Omitted for Anonymous Review

Abstract—When humans learn new manipulation skills, they are able to generalize these skills to new contexts and environments. In particular, when learning, humans can easily separate task-relevant aspects of the environment (e.g., object location) and distractors (e.g., the table color). Ideally, robot policies should be able to learn similar, generalizable representations, but instead they often fail under small environment shifts such as changes in lighting, object instance, or initial configuration. In this paper, we propose a self-supervised method that learns a *mask* which, when multiplied by the observed image features, attempts to *transform* these features to retain only those which are relevant to the task. Our method — which we call *TransMASK* — can be combined with a variety of imitation learning frameworks (such as diffusion policies) without any additional labels or alterations to the loss function. By introducing a learned mask to the network during training, we aim to induce competitive pressure among the image features during training to force the policy to only attend to features which are consistently task-relevant. We find empirically that our masks are interpretable and can reject known spurious features such as the position of distractor objects or the background color. When compared to other representation learning methods for imitation learning, we find that TransMASK results in policies that are more robust to distribution shifts for irrelevant features, achieving at least 30% improvement over the baselines when tested on out-of-distribution environments. See our project website: <https://transmask.github.io/TransMASK/>

Index Terms—Imitation Learning, State Representation, Representation Learning

I. INTRODUCTION

Imitation learning enables robots to learn tasks from offline demonstrations provided by a human expert. Consider teaching a robot to pick up a block and place it at the center of the table (see Figure 1). The human teacher guides the robot through successive states such as reaching the block, grasping it, and placing it at the desired location. In the demonstration, the robot receives observations of the scene recorded through cameras and the actions applied by the expert. When demonstrating the task, the human expert focuses only features crucial for completing the task — the object, the goal, and the robot positions. However, the robot’s observations record information about the entire scene, capturing lighting conditions, table texture, background objects, and other task-irrelevant features.

A policy trained using these observations inherits this issue: it can attend to all parts of the state, including the details a human demonstrator would ignore. This causes the policy to become increasingly brittle. Consider Figure 1: when deploying a policy trained with demonstrations over a wood table for an identical task on a marble table, the visual observations become out-of-distribution, causing the policy to fail.

Our key idea is that a robust policy is one that attends to the same features a human expert would. Thus, we aim

to design policies with structures to mask out task-irrelevant information, enabling the policy to make decisions using features inherent to the task (e.g., object position, end-effector pose) instead of the scene(s) in its training data.

Existing works have developed a variety of methods for extracting this relevant state. A straightforward approach augments the training data with random transformations of visual observations (e.g., random cropping, adding noise to the image) [1], [2]. Nevertheless, introducing variability in the training data can degrade in-domain performance and does not guarantee robustness to large distribution shifts. Another line of work leverages vision-language model (VLM) backbones [3], [4] that have robust state representations due to large-scale pretraining on diverse datasets. Despite these advantages, VLMs typically require task-specific finetuning, which can lead to catastrophic forgetting of pretrained features adversely effecting the robustness. Alternatively, a different class of literature focuses on learning minimal state representations that can extract only the most relevant information from the observations. This is achieved by introducing a trade-off between i) disconnecting the latent state from the actual state (compression) and ii) ensuring the latent state can map to correct actions (performance). Examples of this approach include using variational autoencoders (VAEs) for encoding the images to states [5], [6], training an encoder with the Information-Bottleneck (IB) principle [7], or adding structure to the representation space using contrastive learning [8], [9]. However, these approaches — as we will show empirically and theoretically — rely on an ill-posed optimization problem that often collapses into inaccurate state representations that lack either compression or performance.

We therefore propose a new method for extracting the correct state from observations without modifying the optimization objective. We begin with the intuition that actions demonstrated by the expert are conditioned only on parts of the state that are relevant to the task. When optimizing a standard imitation learning objective, predicted actions will correlate strongly with these elements and weakly correlate with extraneous elements. Consequently, the magnitude of gradients associated with action-relevant elements will be larger. Building on this intuition, we hypothesize that:

The policy Jacobian can be a proxy for causal relevance.

Put simply, we can exploit the gradients to identify and preserve the components of state that matter for control. This prevents the robot policy from learning spurious correlations between states and actions. In order to ensure that the state space is structured, we assume that the robot state is disentangled and can be decomposed into relevant and irrelevant

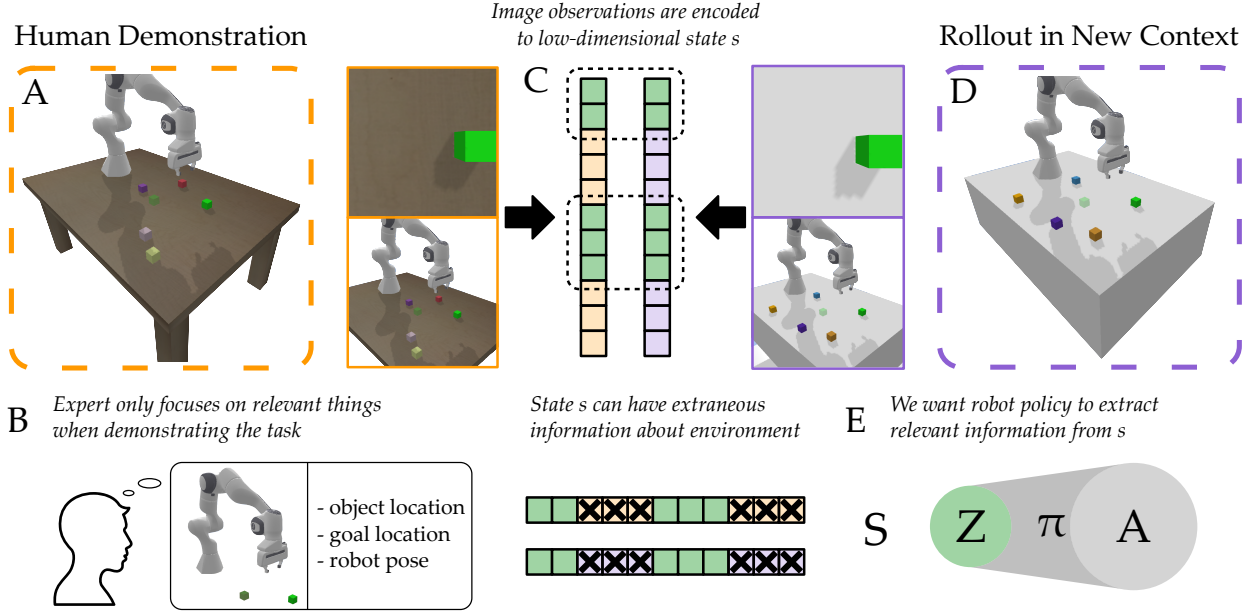


Fig. 1. Robot learning to pick up a green block from a cluttered environment and place it at the center of the table. (A) The expert demonstrations are collected over a wooden table. (B) This expert’s decisions depend only on features intrinsic to the task (e.g., green block, robot position, and target position). The robot’s observations in these demonstrations, however, record information about the entire scene, capturing the table texture, background color, and task-irrelevant objects. (C) We assume a disentangled state s is extracted from these high-dimensional visual observations. In this state, some elements correspond to the task structure — positions of the green block, target, and robot, while others correspond to scene-specific factors. In the figure, relevant elements are shown in green and scene-specific elements are shown in orange and purple. (D) Standard imitation learning policy attends to the entire state. When this policy is deployed for the identical task in a different scene (e.g., over a marble table), it may fail due to spurious dependencies learned during training. (E) To learn a robust policy, we must encode the state to a representation z which masks-out task-irrelevant features. A policy that is conditioned on z is more robust to distribution shift caused by changes in the scene that do not alter the task structure, for instance, when the task is still to pick and place a green block but over a marble table instead of a wooden table.

elements. For example, the first k elements of the state could represent the location of the block and the robot¹. Despite the additional structure imposed on the state space, the individual states can still contain environmental noise which can lead to causal confusion. In our running example, if the environment has other blocks in addition to the green one, the policy can incorrectly map the position of these “distractors” to the action. Consequently, a change in the position of any of these extra blocks can cause the state to go out-of-distribution even when the green block is not moved.

Applying this insight we derive TransMASK, a method that can extract relevant information from the state without additional supervision. TransMASK relies on the underlying optimization scheme of imitation learning and utilizes its gradients to learn a transformation matrix. This matrix transforms the robot’s state into a representation which preserves elements important for action prediction while suppressing the rest. Our approach is a step towards robots that learn robust policies from human demonstrations, and can apply their policies to new testing environments.

Overall, we make the following contributions:

Identifying Why Existing Approaches Fail. We show how — in imitation learning settings — current approaches for state

representations are ill-posed. i) These methods rely on a non-convex optimization that often gets stuck in local minima, ii) even when solved, the latent state can collapse to be just an action representation, iii) there are hyperparameters which are non-trivial to tune for achieving efficient trade-offs.

Deriving TransMASK. We specifically focus on settings where the state is disentangled into relevant and irrelevant elements. Within this setting we develop a state representation method that naturally learns a representation z of the relevant state while purely optimizing for performance. Our approach is linked to attention networks, and we harness the properties of the policy Jacobian to reach our proposed approach.

Comparing to Baselines. We compare our approach to state-of-the-art representation baselines across a variety of simulated and real-world environments with visual as well as privileged state information. TransMASK achieves up to 15% higher performance than the next best baseline when tested in settings without distribution shift and approximately 9% higher success under environmental perturbations.

II. RELATED WORKS

Robust Imitation Learning. Imitation learning has advanced rapidly in recent years, driven by the development of expressive architectures such as diffusion policies [10] and transformer-based policies [11]. Despite these successes, deploying imitation learning policies in scenarios that differ from the training distribution (e.g., different object locations, clutter,

¹We clarify that obtaining such disentangled features for our method does not require additional domain knowledge. As we will show later in our experiments, segmentation masks for all objects present in the scene can satisfy this assumption without the need for additional information from the designer, since the object labels can be simply obtained from a pretrained vision language model without needing finetuning.

or visual features) remains an open challenge [12], [13]. Several works have attempted to address this challenge and make policies robust to distribution shift, out-of-distribution observations, or noisy observations. Broadly, these works focus on two distinct approaches to achieve robustness: one set of methods focus on the *training data* while another focuses on the *training procedure*.

Data curation and diversification: The first class of methods focus on structuring the demonstration dataset so that the learned policy attends to the most relevant aspects of the state space. A straightforward strategy is to collect datasets while ensuring data diversity to improve robustness [14], [15]. However, in many practical settings, we may have limited control over data collection, since it is common to use readily available datasets such as [16], [17]. Therefore, several approaches aim to re-balance the given dataset to learn diverse behaviors reliably [18], [19]. Other works improve generalization through domain randomization [1], [2], exposing the policy to variations in the observations during training. Alternatively, we can augment the available dataset with auxiliary data to improve robustness of the trained policy. For example, [20], [21] leverage auxiliary datasets alongside optimal data to improve performance, while [22], [23] learn a dynamics model from expert data and use it to generate additional synthetic demonstrations.

Adversarial BC: The second class of methods modifies the training pipeline to enhance robustness. This is typically achieved through adversarial training which improves model robustness by minimizing the prediction loss under worst-case perturbations of the input. In the context of imitation learning this is formulated as a generative adversarial game in which the policy is trained jointly with a discriminator that distinguishes between expert trajectories and those generated by the policy. The most prominent works in this domain include GAIL [24] and f-GAIL [25], while other works have extended this idea to achieve better sample efficiency and performance [26], [27].

While these approaches in imitation learning have been shown to improve performance and robustness, they do not address the more fundamental problem: we should condition the policy only on the relevant parts of the state. Data curation approaches can have computational overheads related to generating additional data or finding the right balance for the demonstration’s prior to training the policy. Some of these methods rely on a human expert for labeling the dataset which can be time consuming. Adversarial training can often introduce instability in optimization, requiring precisely tuning the discriminator, and is often unsuitable for high-dimensional inputs such as visual observations. Additionally, variants of GAIL are sample-inefficient as they handle distributional shift by interacting with the environment.

State Representations. There are various works that explicitly focus on extracting relevant information about the task to achieve robust performance by encoding the observations to a structured representation space. Recent advances in object-centric representations, such as VIMA [28] and Transporter Nets [29], decompose the scene into object tokens using segmentation. While this has shown to improve spatial reasoning,

they attend to all discovered entities equally. A more general approach requires compressing the input by retaining only critical information from the state and discarding redundant or unnecessary features. Contrastive learning, which optimizes the latent representation space by minimizing the distance between semantically similar samples while maximizing the distance between dissimilar samples, has emerged as a viable solution to this problem. In imitation learning, contrastive loss is leveraged to learn representations that are robust to variations in visual observations [8], [30]. While [30] uses auxiliary static observations of the robot alongside demonstrations and identify positive and negative pairs using temporal proximity of the observations, [8] uses action sequence similarity to identify positive and negative pairs.

However, a contrastive approach typically requires comparing each representation with many negative examples to prevent representation collapse. An alternative is to use the Bootstrap Your Own Latent (BYOL) objective [31] to learn invariant state representation from image observations as in [32]. A simpler approach is to leverage self-supervised training using variational autoencoder (VAE) for extracting representations. For example, [33], [34] use VAE to extract motion primitives from robot trajectories; and [5], [35] use VAE to extract better state representations from image observations.

Learning state representation has also been viewed from an information theoretic perspective, where mutual information can eliminate redundancy in the learned representations. Specifically, research has examined the use of the Information Bottleneck (IB) principle [7] for deep representation learning, demonstrating that IB can make latent representations invariant to irrelevant parts of the inputs [36], [37]. This principle has also been leveraged for learning task-relevant state representations for robust imitation learning [38].

Despite their success, there are limitations to these approaches. First, contrastive learning requires identifying positive and negative samples in the demonstrations which requires additional domain knowledge. Training can also be unstable in contrastive learning resulting in representation collapse. Further, approaches like VAEs can encode high-dimensional visual observations to a structured representation space, but they do not explicitly aim to retain only task-relevant information. Finally, while the IB seeks to compress the state, in the context of imitation learning, it often lacks an explicit supervision signal to distinguish between nuisance factors and features important for task completion. Later in the text, we will provide a detailed discussion on why the IB objective is ill-posed for imitation learning applications. In contrast, TransMASK extracts task-relevant features as a byproduct of the imitation learning gradient flow, alleviating the need for additional data or modifying the training pipeline and the objective function for optimization.

III. PROBLEM STATEMENT

We consider offline imitation learning settings. Within these settings, the robot is given a fixed dataset of demonstrations. In each demonstration a human expert teleoperates the robot through instances of the desired task. Based on these examples

the robot learns a control policy that can robustly perform the demonstrated task across a variety of initial states. In particular, the learned control policy should be robust to changes in the scene that do not alter the task (e.g., if the color of a table changes, or if clutter is added to the background). We specifically focus on settings where the state is *disentangled*. There are some elements of the state that are relevant to the expert’s control decisions, while other state elements are irrelevant noise that do not impact how the expert behaves. To separate the relevant and irrelevant state elements, we introduce a latent state representation $z \in \mathbb{R}^d$.

Dataset. Each demonstration consists of a sequence of tuples $\xi = \{(x^1, I^1, a^1), \dots, (x^t, I^t, a^t)\}$ where $x \in \mathbb{R}^m$ is the proprioceptive state (e.g., joint angles), $a \in \mathbb{R}^m$ is the applied action (e.g., joint velocities), and I is the robot’s observation (e.g., images captured from onboard and/or static camera(s)). Overall, the dataset includes a few demonstrations for the desired task. In our settings, we consider state $s \in \mathbb{R}^n$ which includes the proprioceptive states x as well as features ϕ encoded from image observations I , i.e., $s = [x, \Phi(I)]$ using visual encoder Φ . For every state in the dataset, the human expert labels what action the robot should take in order to complete their desired task.

Human. The expert’s policy π^* , which they use to provide demonstrations, only considers task-relevant features and is not influenced by scene-specific components. For example, when demonstrating how to pick up a block, the human only focuses on the block’s location and the robot’s location relative to the block. Their decisions are not influenced by texture of the table, lighting conditions, or the presence of clutter². However, the state s that the robot records when the human is demonstrating the task includes all the information from the scene.

Disentangled State. In our problem setting, we assume that the state vector s is disentangled and can be separated into two components:

$$s = [s_1, \dots, s_n]^T, \quad s_i \in \mu \text{ or } s_i \in \eta \quad (1)$$

Here $\mu \in \mathbb{R}^k$ are the *relevant* elements of the state that the human expert uses to make their control decisions. Returning to our running example, μ could contain the location of the block. The remaining state elements $\eta \in \mathbb{R}^{n-k}$ are *irrelevant* to the expert’s control decisions, and can be treated as extraneous information. For example, perhaps η includes the color of the table or the location of background objects.

We emphasize that the *disentanglement* of state s is fundamental to our proposed approach. We argue that this sort of separation is often inherent to the learning context (e.g., humans only focus on a few relevant features when making decisions [39]). However, we recognize that the state s that the robot extracts from the scene may not align with these disentangled representations, and we will test the limits of this assumption in our experiments.

²Provided that such clutter does not interfere with grasp feasibility. In our setting, clutter is treated as a potential source of spurious correlation or causal confusion, rather than an obstruction to manipulation.

Latent State. Since the state s can contain irrelevant information, we do not want our robot learner to make decisions based on every aspect of this state. To make the policy robust, it should focus only on task-specific information μ . However, we do not know which elements of the state constitute μ and which elements constitute η . Therefore, we seek to learn a state representation $z \in \mathbb{R}^n$ that only encodes the elements of s that strongly correlate with the expert’s demonstrations. We instantiate this encoding as a *state mapping* $z = f(s)$. Ideally, we want to optimize the following objective:

$$f_\theta \in \arg \min_{\theta} \mathbb{E}_{s \in \mathcal{D}} [H(\mu | f_\theta(s)) - H(\eta | f_\theta(s))] \quad (2)$$

Here $H(a | b)$ is the conditional entropy (i.e., the uncertainty) over variable a given variable b [40], and f is parameterized by weights θ . Intuitively, if μ and z are correlated, then $H(\mu | z) \rightarrow 0$ because we can determine exactly what μ is after $z = f(s)$ is measured. Conversely, if η and z are uncorrelated, then $H(\eta | z) \rightarrow H(\eta)$ because z does not provide any information about η . Overall, Equation (2) formalizes the type of state mapping f_θ we want to achieve — the state mapping should extract μ from s while disregarding η . However, we cannot directly implement Equation (2) because the robot does not know which state elements are relevant or irrelevant *a priori*. Our approach will aim to optimize this objective implicitly using the supervision available in the imitation learning loss.

Policy. Using the offline dataset $\mathcal{D}$ and the state mapping f_θ , the robot learns a control policy that automates the demonstrated task. Let $a = \pi^*(\mu)$ be the expert policy that outputs the correct action a for each state s . We seek to imitate this expert policy through a compositional approach: first, the state mapping $z = f_\theta(s)$ encodes the latent state, and second, our learned policy $a = \pi(z)$ maps that latent state into robot actions. We write our overall policy as $a = \pi_\psi(f_\theta(s))$ with model weights ψ and θ . Our objective is for this compositional policy to achieve two things. To learn performant behaviors, the actions $a = \pi(z)$ should match the expert’s demonstrated actions across dataset $\mathcal{D}$. To increase robustness to environmental variations, the latent $z = f_\theta(s)$ should retain task-relevant information μ and discard extraneous elements η .

IV. TRANSMASK

We want the robot to learn tasks from human demonstration while being robust to environment variations. As previously discussed, only the task relevant features are factored into the expert’s decision making, and a robust robot policy should only be conditioned on these aspects of the state μ . To extract μ , we propose a method to learn state representations $z = f(s)$ that fulfill two desired characteristics — it must retain information from s that enables the policy to match the actions demonstrated by the human, while removing information η that does not relate to the task structure. In this section, we start by demonstrating how standard approaches for learning latent representations — such as autoencoders — are poorly designed for this objective (Section IV-A). Next, we analyze the Jacobian of the expert policy to determine the characteristics of an effective state representation (Section IV-B).

Finally, we derive our proposed framework for learning z , and prove that this approach addresses the issues with standard representation techniques (Section IV-C).

A. State Representation and Collapse

In Equation (2) we present an objective that can be optimized to learn a state representation z which only extracts task-critical information from the state while excluding unimportant information. As mentioned earlier, we cannot directly optimize this objective because the relevant and irrelevant parts of the state are not known a priori. However, we know that while demonstrating the task, the human expert’s actions are not influenced by some parts of the state. Taking this into consideration, we can reformulate the objective from an information theoretic perspective and use mutual information to quantify the relevance of the elements in the state. Specifically, we can rewrite the objective from Equation (2) in terms of state s , action a , and state representation z where we want to maximize $\mathcal{I}(f(s), a)$ and minimize $\mathcal{I}(s, f(s))$. Here, $\mathcal{I}(A, B)$ is the mutual information between A and B , measuring the amount of information gained about one random variable by knowing the other. The commonly used framework in previous literature for such an objective is the Information Bottleneck (IB) principle [7]:

$$\begin{aligned} L &= \lambda \mathcal{I}(s, z) - \mathcal{I}(z, a) \\ &= H(a | z) - \lambda H(s | z) \end{aligned} \quad (3)$$

We derive the second step by using the definition of mutual information $\mathcal{I}(A, B) = H(A) - H(A | B)$, and $z = f(s)$. Equivalently, the loss can be re-written as:

$$\max H(s | z) \quad \text{subject to} \quad D_{KL}(\pi^*, \pi) \leq \lambda \quad (4)$$

This objective can be broken down into two loss functions — a performance loss function that encourages the model output to match the expert dataset and a regularization loss that serves to disconnect z from s . Together these terms can ensure that the state representation only extracts the information from s which is necessary for accurately predicting the expert action. Different approaches utilize different regularization: for example, [38] estimates the mutual information $\mathcal{I}(z, s)$ to learn reliable robot policies, while others such as [41] use a VAE to constrain the representation space to be close to a unit Normal distribution. *However, this framework is ill-posed due to four fundamental issues.*

First, this type of objective with both a minimizing and a maximizing term is tricky to optimize due to the presence of multiple optima as well as the instability introduced by the conflicting terms. Second, these methods require the designer to choose λ “by hand” to balance the loss terms, and this heuristic tuning can be challenging. If we decrease λ too much, then we are overly focused on performance, and may learn a latent state z that matches the original state s , which is counter to our goal of extracting only relevant information from s . By contrast, if we increase λ too much, then we are overly focused on compression, and may learn a latent z that is completely separated from s — harming our ability to actually learn and perform the task. Third, estimating the

mutual information between s and z can be intractable for most applications. Finally, when applied to imitation learning contexts, these structures allow z to collapse into an action representation rather than a state representation.

To demonstrate this collapse, consider a simple network architecture with an encoder that maps s to z and a policy which maps z to a . The architecture can be trained by optimizing the reconstruction loss on action a . This simple architecture is a straightforward, albeit naive, approach to implementing the optimization scheme presented above. In this simple case, z could simply be a sufficient encoding for reconstructing the action accurately, but it may not necessarily extract relevant information from s . Put simply, z learns a latent space that is an action representation and not a state representation. This issue arises because the optimization scheme of VAEs does not explicitly incentivize it to encode state information, rather it forces the decoder to predict the action given a latent representation. As a result, the latent space can collapse into a minimal sufficient statistic of s for predicting a . This may exclude semantic information from the visual observations, such as task object, goal location, and even the distractors.

One way to force z to be a representation of the state is to add a decoder that maps z back to the state s , i.e., a typical autoencoder. This structure does prevent the representation collapse described above. However, the latent space learned by this autoencoder embeds all the elements of s — relevant μ as well as irrelevant η . Indeed, if the irrelevant features fluctuate more across the dataset than the relevant features, then z is likely to emphasize these irrelevant features in order to accurately reconstruct s , again leading to failure.

B. Jacobian of the Expert Policy

Instead of using the ill-posed optimization of the IB principle for imitation learning, we intend to optimize for performance alone and utilize the underlying gradient structure to extract a compact representation of the state. We start by taking a closer look at the expert policy $\pi^*(s)$. Remember, in our problem settings we assume that the state $s = (\mu, \eta)$ is composed of relevant and irrelevant elements, and these elements are disentangled, i.e., μ and η are mutually exclusive and exhaustive ($\mu \cap \eta = \emptyset$, $\mu \cup \eta = s$). With this in mind, consider the Jacobian of the expert policy $\nabla_s \pi^*(s)$. This Jacobian is a $m \times n$ matrix, where m is the dimension of the action a and n is the dimension of the state s . The columns of this matrix indicate the impact of the corresponding element of the state on the robot’s actions, or in other words, the sensitivity of the action to the corresponding element of state. For example, the first column indicates how the first element of s affects each dimension of a . Since the expert’s decisions are not influenced by the irrelevant elements of s , the column i of the Jacobian corresponding to $s_i \in \eta$ will be all zeros. In contrast, the columns j of the Jacobian corresponding to $s_j \in \mu$ will have non-zero values since these elements affect the expert’s actions. Overall, $\nabla_s \pi^*(s)$ will be a sparse matrix such that the states that contain purely irrelevant information are in the *null space* of the Jacobian. In the next subsection, we discuss how this property helps us learn a state representation that extracts only the relevant information from the state.

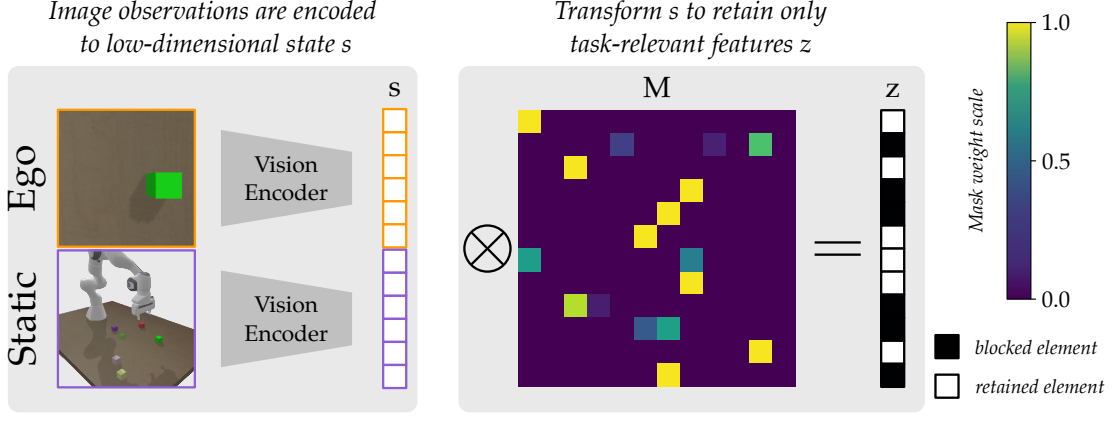


Fig. 2. Schematic diagram of TransMASK. The visual observations are encoded to a disentangled state vector s . We introduce a mask encoder that outputs a constant mask M of shape $n \times n$, where n is the dimension of s . This mask is a sparse matrix in which the columns that correspond to task-irrelevant elements of s are close to zero. Therefore, when we compute the representation z by transforming the state with M , it only retains the elements critical for accurately predicting actions. The robot policy is conditioned on z rather than the entire state to achieve robustness.

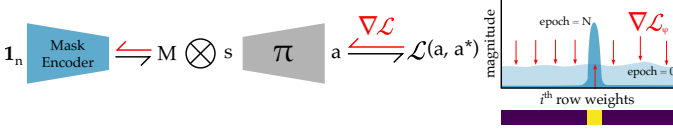


Fig. 3. Overview of how the mask is learned. (Left) The mask encoder inputs a 1-vector of size k ($\mathbf{1}_k = [1, 1, \dots, 1] \in \mathbb{R}^k$) and outputs a matrix $M \in \mathbb{R}^{n \times n}$. Since the expert's actions when providing demonstrations are only influenced by the task-relevant features in the environment, the columns of the Jacobian of the expert policy corresponding to the irrelevant elements in the state will be near-zero magnitude as discussed in Section IV-B. (Right) When training the robot policy, as the loss converges the Jacobian of the robot's learned policy changes the rows of M until the values start weighting the task-relevant elements more than the irrelevant ones.

C. Deriving TransMASK

Following our discussion about the pitfalls of the IB objective and the Jacobian of the expert actions, we now discuss how we can learn a correct state representation. The fundamental problem with the IB objective arises due to conflicting objectives — performance and regularization. We aim to eliminate the regularization loss term from the objective. As the Jacobian of the action emerges naturally when optimizing the imitation learning objective (performance term), we can leverage the Jacobian to learn a state representation without the need for regularization. To this end, we define state representation z as an affine transformation of state, i.e. $z = Ms$, where M is a $n \times n$ square matrix. We will refer to M as a *mask*, since it can mask-out or remove elements of s from the resulting z . Indeed, each element of z here is a linear combination of the state elements, i.e., $z_i = M_{i1}s_1 + \dots + M_{in}s_n$. Selecting $z = Ms$, where M is a constant matrix, ensures that the state representation maintains a correlation with the state and does not collapse into an action representation. This also eliminates the need for estimating the mutual information between s and z , which is often intractable. Further, as we will show later, the regularization loss emerges naturally as part of the training and the structure of the model. Now, we only need a performance loss to encourage the model to predict accurate actions given a state. The natural choice is the MSE loss which is commonly

used in behavior cloning:

$$\mathcal{L}(\psi, M) = \sum_{(s,a) \in \mathcal{D}} \frac{1}{2} \|\pi_\psi(Ms) - a\|^2 \quad (5)$$

Since the matrix M is constant, we parameterize it with a learnable parameter θ . The state s , which contains disentangled features obtained from image observations, is transformed into a representation $z = Ms$. As a result, our approach is versatile because the mask can be easily appended to any existing IL policy without augmenting the training process or the loss function. We show later in our experiments that this enables the network to learn a mask that only focuses on parts of the state that relate to the task structure μ while removing the unimportant elements η . Our method is summarized in Figure 2.

To demonstrate how the Jacobian can naturally learn such a mask, we take the gradient of our loss in Equation (5) with respect to the mask weights θ :

$$\begin{aligned} \frac{\delta \mathcal{L}}{\delta \theta} &= \left(\frac{\delta(\pi(z) - a)^T (\pi(z) - a)}{\delta M} \right)^T \frac{\delta M}{\delta \theta} \\ &= s \frac{\delta(\pi(z) - a)^T (\pi(z) - a)}{\delta z} \frac{\delta M}{\delta \theta} \\ &= s \cdot (\pi(z) - a)^T \cdot J_{\pi z} \cdot \frac{\delta M}{\delta \theta} \end{aligned}$$

Here, we apply the vector chain rule through the computational graph, and $J_{\pi z}$ is the Jacobian of the policy with respect to z . If M is a learnable parameter matrix $\theta \in \mathbb{R}^{n \times n}$, this gradient of the loss is an $n \times n$ matrix. On the other hand, if M is the output of a neural network with parameters θ , the gradient of the loss becomes a tensor of shape $n \times n \times p$, where p is the input dimension of the network.

As the policy model trains, we assume that the robot policy π approaches the expert policy π^* , which implicitly requires their Jacobians to align ($\nabla_s \pi \rightarrow \nabla_s \pi^*$). For task relevant elements ($s_i \in \mu$), changes in input directly alter action predictions to minimize the loss. Consequently, these elements receive high magnitude gradients $\|\mathcal{L}/\delta \theta_i\| \gg 0$ leading to continuous growth in their corresponding mask

elements. Conversely, task-irrelevant elements ($s_j \in \eta$) do not influence the expert’s action, so their corresponding mask elements receive negligible gradients $\|\mathcal{L}/\delta\theta_j\| \approx 0$ and remain largely static. To ensure that the mask M acts as a feature selector, we apply row-wise normalization, e.g. softmax, to the mask matrix. The exponentiation inside softmax creates a competitive pressure across each row of M . Concretely, the exponential of the elements in M corresponding to high gradient parameters θ_i dominate the denominator, suppressing the values of elements corresponding to θ_j . Ultimately, the relevant mask values are driven up to 1 while the irrelevant mask values are suppressed to 0. Put simply, the mask matrix attains a sparse structure wherein the columns corresponding to important features of the state ($s_i \in \mu$) are non-zero, while the columns corresponding to environment noise ($s_j \in \eta$) are largely zero. As a result, the learned mask M acts as a feature selector when multiplied with the state s by filtering out task-irrelevant features while retaining the relevant ones.

D. Comparing with Attention Mechanism

Here we provide an alternative perspective on our proposed TransMASK framework. Our mask can be viewed as an attention mechanism [42] where we learn the attention weights for the elements of the input state; the state representation is the matrix product of the state with these learned attention weights. In attention mechanisms, the attention weights for the token pairs is defined by the formula: $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$. During training, there is no direct supervision for learning the correct attention weights. Rather, the back propagation of gradients from the loss push query Q and key K to produce a larger dot product for the important pairs and lower dot product for unimportant pairs. As a result, the networks that learn the key, query, and value are updated such that the model is able to discover the correct attention patterns that minimize the loss. Similar to this, in our method there is no direct supervision for training the mask. Rather, it is learned as a result of the gradients that encourage the mask elements to focus more on the parts of s which influence the output while ignoring the irrelevant parts.

Nevertheless, our method fundamentally differs from conventional attention mechanism in the way it weights the elements of the state. Attention mechanisms are designed to model the relative importance of different components of the input, whether these correspond to different elements in a sequence or spatial patches of images. In contrast, our task requires transforming a single disentangled state vector into a compressed representation that retains only the task-relevant elements. At first glance, attention might appear to be a natural solution. However, a standard attention mechanism computes dynamic attention weights that are conditioned on the input, i.e., the key, query, and value are computed via neural networks that take the state vector as input. While this allows the model to focus adaptively on different elements of the state, it also introduces the risk of losing the original disentanglement as task-irrelevant elements may be inadvertently weighted and propagated through the representation. In contrast, our method uses the policy Jacobian as supervision for feature selection.

As a result, we learn a static mask that is input-invariant as it is independent of the state. This static mask reflects our underlying assumption about the state decomposition into μ and η . This design is particularly advantageous because, even when deploying a policy in a new environment, task relevance should be determined by task structure, rather than the variations in the input values. For instance, in a pick-and-place task, the end-effector and object poses are always relevant, while visual textures, or background color are generally not relevant. Thus, making our attention mask input-dependent, as in the conventional attention mechanism, would simply provide another path to allow task-irrelevant information to enter the policy.

To summarize, our method shares some similarity with the attention mechanism in the learning dynamics — the way attention mechanism is able to learn the attention weights is closely related to how our mask learns which elements of the state to attend to. However, it has a principal distinction from the attention mechanism. TransMASK learns a static, state independent mask leveraging the disentangled structure of the state, transforming it into a meaningful state representation for downstream policies.

V. EXPERIMENTS

TransMASK learns a mask that attends only to the elements of the state that are relevant for performing the given task. In this section, our goal is to validate the proposed approach and test whether it can achieve robustness against data which is outside the training distribution. We conduct extensive experiments in simulation as well as real-world tasks. Additionally, we incorporate our method with two policy classes: MLP, which uses fully-connected architecture as the policy head; and Diffusion Policy (DP), which is a policy head trained with the diffusion objective [10].

Baselines. Across our experiments, we compare TransMASK with state-of-the-art state representation baselines. These baselines learn well-structured representations for the state either through pretrained visual encoders, contrastive learning, or by imposing a prior on the representation space. Below we discuss these baselines in more detail.

- **Behavior Cloning (BC).** [43] A standard imitation learning approach. BC learns to map the state to robot actions as demonstrated by the expert. This method does not explicitly encode the state to extract task-relevant information; rather, it attends to the entire state when predicting robot actions.
- **Variational Autoencoder (VAE).** [44] A self-supervised approach that learns to encode the state into a structured representation space. The decoder is modified to act as the policy head and maps the encoded feature to the expert action. This architecture introduces a bottleneck that tries to force the robot to retain only critical information.
- **CLASS.** [8] A supervised contrastive learning approach. It aligns observations in the representation space based on the similarity of the future action sequences. This allows the robot to learn shared structure across demonstrations.

- **VINN.** [32] A self-supervised approach that separates representation learning and policy learning. The visual encoder is pretrained using the Bootstrap Your Own Latent (BYOL) [31] objective. For new observations, it finds the nearest neighbors in the representation space and chooses the weighted average of the actions corresponding to the neighbors.

A. Simulations

First, we test our method against baselines in a simulated environment. The robot is trained on demonstrations collected in one scenario. We test if TransMASK can retain good performance when encountering visual perturbations like changes in the lighting conditions, background variations, etc.

Simulation Environment. We train and evaluate all methods on three tasks in the Panda-Gym environment [45]. The environment includes a 7 degrees of freedom (DOF) Franka Emika robot arm mounted on a table. Different colored blocks are scattered in the robot’s workspace during data collection and evaluation. These blocks are randomly initialized within a circle of radius 0.1 meter in the center of the table. During evaluation, we test the methods on in-distribution data, where the environment scene is the same as it was during data collection, and on out-of-distribution data with significant visual variations. The visual perturbations are introduced by changing the position and intensity of the lighting source, changing the table on which the robot is mounted, and changing the appearance of the blocks. The left columns in Figure 4 display the two scenes. The figure on top shows the robot setup used for data collection and in-distribution evaluation, while the figure on the bottom shows the setup used for out-of-distribution evaluation. Demonstrations are collected using a simulated human teacher and we record the ground truth pose of all objects from the simulation environment.

We design three simulated tasks: picking up and placing a block at a target location (*Pick*), pushing a block to a target location (*Push*), and rotating a Rubik’s cube to a desired orientation (*Rotate*). In *Pick* and *Push* tasks, the environment is initialized with a green block that the robot needs to move to a target location. This block along with the target location is randomly initialized each time. Additionally, 4 different colored blocks are also initialized to act as distractors — the robot should learn that only the green block and the target location are the relevant features in the state, and therefore the robot can ignore the irrelevant information, i.e., the state of the distractor blocks. In *Rotate*, the environment is initialized with 5 Rubik’s cube 4 of which have randomly initialized orientation while the desired cube always starts with the same orientation. The location of each Rubik’s cube is randomly selected at the start of each episode. The robot must pick and rotate the cube by 90° about the longitudinal and vertical axes. The robot should learn to pick up the target cube with the same orientation while ignoring the other cubes and table color.

Demonstrations. At each timestep of the demonstration we collect an RGB image I_{env} of size 256×256 from a static camera that observes the entire simulated environment, an ego-centric image I_{ego} captured from a camera mounted on the

robot’s end-effector of size 256×256 , the robot’s state x , and the robot’s action a applied by the expert human. In *Pick* and *Push*, x is an 8-dimensional vector including end-effector pose and a boolean gripper state, while a is a 7-dimensional vector including linear and angular velocity of the end-effector and the gripper actuation. In *Rotate*, x is an 8-dimensional vector including the joint angles of the robot and a boolean gripper state, while a is also an 8-dimensional vector including the joint velocities and gripper actuation. This target pose is the Euclidean position where the block should be moved in *Pick* and *Push*; while in *Rotate* it is the desired orientation that the cube must obtain. Additionally, we extract segmentation masks of all the blocks, the robot, the gripper, and the table in the form of bounding boxes $\{\mathcal{B}\}$. The coordinates of the bounding boxes are concatenated into a segmentation feature ϕ_{seg} of dimension 32. This segmentation feature satisfies our assumption that the state is disentangled — only the bounding boxes of the robot and object are relevant for the task.

Procedure. Since the observations include high-dimensional images, we first use a ResNet-18 vision encoder to map the images I_{env}, I_{ego} to a low-dimensional space ϕ_{env}, ϕ_{ego} . The state includes these image features, the robot state, and the segmentation feature $s = [\phi_{env}^T, \phi_{ego}^T, \phi_{seg}^T, x^T]$. Our method transforms this state into a state representation z retaining only the relevant information needed to complete the task. Ideally, z should focus on the robot state and segmentation features corresponding to the target object, i.e., $z = [x, \phi'_{seg} \subset \phi_{seg}]$. For a fair comparison, all the baselines receive the same state s as inputs.

We train the policies on 100, 100, and 150 demonstrations for the *Pick*, *Push*, and *Rotate* tasks, respectively. We highlight that TransMASK, BC, and VAE are trained only on demonstrations collected in the first robot scene shown in the top left corner of Figure 4. Since VINN and CLASS are designed to reduce the discrepancy between the image features of different observations with similar behaviors, we train these two baselines on demonstrations that are collected from both the robot scenes. *Concretely, VINN and CLASS are trained on datasets with half of the demonstrations collected from the first robot scene and half from the second scene. Our procedure provides an advantage to VINN and CLASS, since they see examples with both types of tables.*

Results. Figure 4 summarizes the results of our simulated experiments. All methods are evaluated for 100 rollouts each in two robot scenes. In the first scene: In-Distribution (ID), which is shown in the top left corner of Figure 4, the observations are from the same distribution as the training set. In the second scene: Out-of-Distribution (OOD), which is shown in the bottom left corner of Figure 4, there is significant distributional shift in the visual observations as the robot is mounted on a different table and the lighting conditions vary each rollout. **Importantly, the observations in OOD are truly out-of-distribution only for TransMASK, BC, and VAE, because VINN and CLASS are trained on demonstrations from both these scenes.** The top row of Figure 4 shows the success rate in ID and the bottom row shows the success rate in OOD.

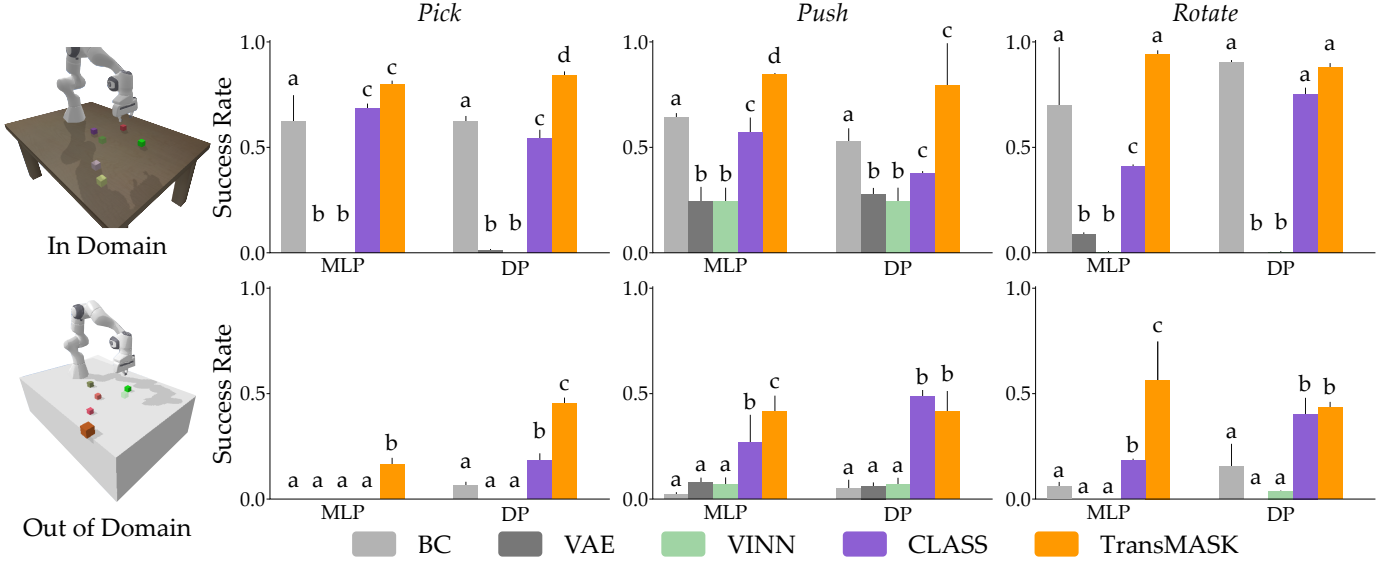


Fig. 4. Results from our simulated experiments. We perform the simulated experiments in three tasks — *Pick*, *Push*, and *Rotate*. All the methods are trained on demonstrations collected over a wooden table, except VINN and CLASS which are trained on a mixed dataset with equal number of demonstrations collected over a wooden table and a marble table. The methods are trained with two different policy heads: a fully connected policy (MLP) and a diffusion policy (DP). We evaluate the methods in two scenes, denoted as In-Distribution or ID (over wooden table, top row), and Out-of-Distribution or OOD (over marble table, bottom row). We use the Compact Letter Display (CLD) algorithm [46] which assigns letters to each policy. The policies which do not share a letter have statistically significant difference in their results.

In each plot, we compare the methods with an MLP action head on the left and a DP action head on the right.

1) *Can TransMASK achieve robustness against visual perturbations?*: In the ID scene, TransMASK consistently outperforms the baselines across the three tasks. This trend holds for both action heads, MLP and DP. One plausible explanation for the effectiveness of masking even in the ID scene is the presence of distractor objects. Standard imitation learning policies can form spurious correlations with the clutter in the scene, leading to causal confusion. Since the target object and distractors are randomly sampled in our setup, the joint configuration of distractors may vary significantly and become out-of-distribution if the policy fails to ignore them.

In OOD scenes, there is significant variation in the lighting conditions — the location, intensity, and color of the light source varies every episode — which causes the visual features to experience a prominent distribution shift. As a result, all policies experience a performance drop of at least 45%. However, our method is able to significantly outperform the baselines across all the tasks for both MLP and DP policy heads. Although there is an exception, the DP policy of TransMASK slightly underperforms compared to CLASS in *Push*, a repeated measures ANOVA test found that this difference is not statistically significant. A likely explanation for why the performance of TransMASK in OOD scene almost reduces to half the original performance is the inclusion of ResNet features which violates the assumption of state disentanglement. While TransMASK suppresses the majority of the ResNet features, some elements retain small, non-zero magnitudes. Since ResNet features are likely to shift drastically with OOD image observations, these residual values negatively affect performance.

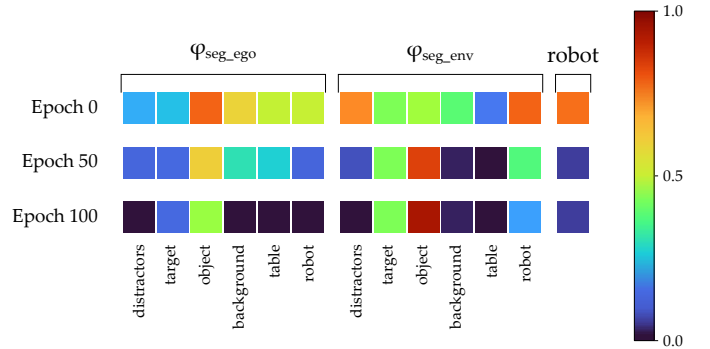


Fig. 5. Learned mask over the course of training. Our method learns a mask $M \in \mathbb{R}^{n \times n}$, and computes the latent state as $z = Ms$. Therefore, the magnitude of the i -th column of M corresponds to the i -th element of s . As the robot policy converges, the values in the columns of M which correspond to task-irrelevant elements of s should ideally approach zero. Here, we show how the weights of the state change as the policy training progresses. We label which components of the column relate to which information in the scene. The components with a dark blue shade have values closer to 0, indicating that the corresponding feature of the scene is removed from z . In contrast, the components with a red shade have values closer to 1, indicating that the associated feature is retained in z .

2) *Does TransMASK learn correct masks?*: After examining the results, we visualize how TransMASK learns to weight the elements of the states. To this end, we visualize the mask M over the course of training in Figure 5. M is an $n \times n$ matrix where n is the dimension of the state. To visualize how the mask assigns importance to different elements of the state, we normalize each column of M . Specifically, for each column, we compute the sum of all its elements and rescale the resulting column such that its values lie in the range $[0, 1]$. These normalized values of the columns indicate which elements of the state are allowed to pass to the state

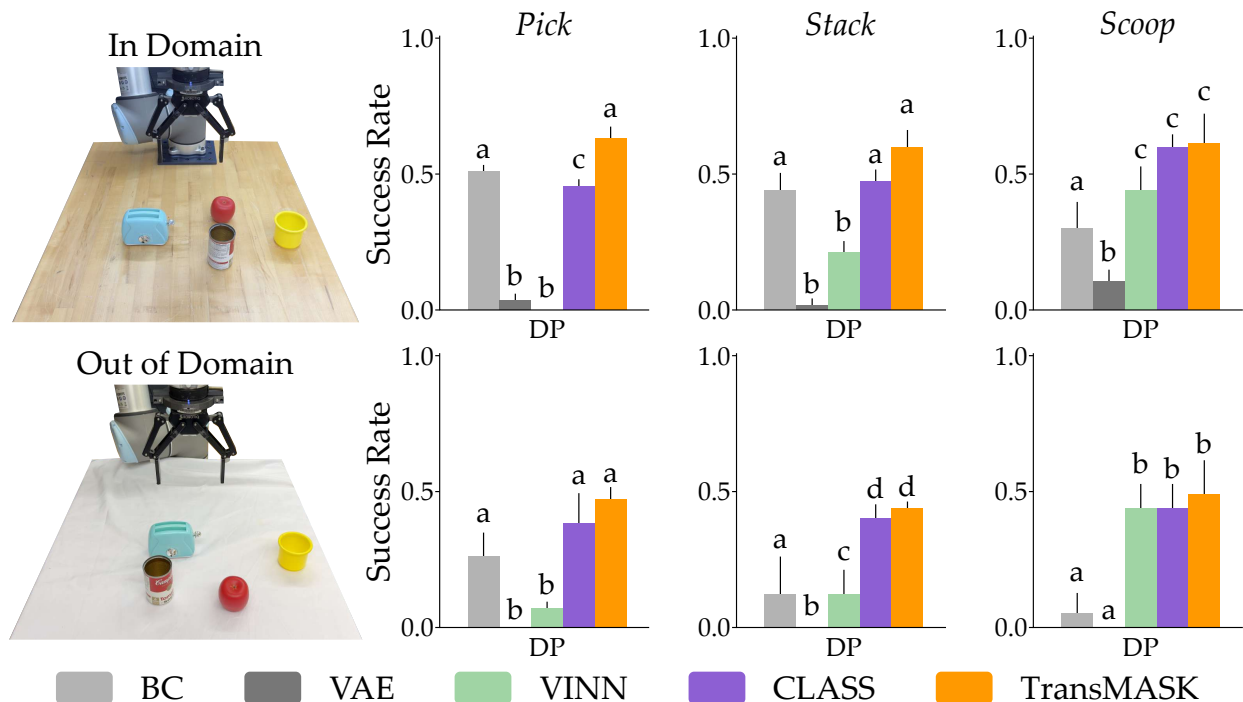


Fig. 6. Results of our real-world experiments. We test the methods in three tasks — *Pick*, *Stack*, and *Scoop*. We evaluate the methods in two scenes, denoted as In-Distribution or ID (over wooden table), Out-of-Distribution or OOD (the table is covered with a white sheet). All the methods are trained on demonstrations collected over the wooden table, except VINN and CLASS which are trained on a mixed dataset with equal number of demonstrations collected from ID and OOD scenes. The policies which do not share a letter have statistically significant difference in the results.

representation z . The normalized values of the columns are close to zero for elements of s that are identified as irrelevant and are not included in z .

Figure 5 shows how the weights of the state change over the course of training. In the beginning, since the mask network is initialized randomly, the weights are almost uniform. This means that the mask allows all the elements of the state to pass through in the representation z . However, with further training the mask learns to selectively filter out elements that correspond to environment noise. Concretely, the weights corresponding to the robot proprioceptive states and the bounding boxes of the object, the robot, and the target in the segmentation features ϕ_{seg} stay non-zero while those corresponding to the bounding boxes of the table, the robot, and the background are slowly pushed towards zero. Interestingly, this is found to be true for segmentation features obtained from both the static camera as well as the robot mounted camera.

B. Real-World Experiments

Next, we move to a real-world setting where a robot arm performs table-top manipulation tasks. Unlike our simulation environment, the images in a real-world setting present unexpected challenges: the objects have textures and the robot may not segment the objects consistently and accurately. Our objective is to test if our proposed method can still be effective in this imperfect scenario.

Experiment Setup. We train and evaluate all methods on a 6 DOF UR10 (Universal Robots) robot mounted on a

table. We use two Logitech C920 cameras to capture image observations. One camera is mounted on the robot’s end-effector to record the ego-centric images and the other camera is mounted on the table such that it observes the entire robot environment. At the start of each task we place the desired object in the environment which the robot needs to manipulate. Additionally, we place other objects in the environment as distractors that can confuse the robot. We want to test whether the robot learns to ignore the clutter in the environment and focus only on the object it needs to manipulate. Similar to the simulated tasks, during evaluation we consider two conditions: in-distribution data, where the scene is the same as during data collection; and out-of-distribution data where we cover the table and the background with a sheet.

We test our method in three tasks: picking up and placing a block at a target location (*Pick*), stacking two LEGO blocks (*Stack*), and scooping from a bowl placed on the table (*Scoop*). In all tasks, the target object is initialized within a predefined region on the left side of the robot workspace, with 2-3 additional objects randomly placed on the table as distractors.

We compare TransMASK against the same baselines used in the simulated experiments — BC, VAE, VINN, and CLASS. We train each method on 20, 25, and 20 demonstrations for the *Pick*, *Stack*, and *Scoop* tasks, respectively. All real-world experiments are conducted using only a diffusion policy (DP).

Demonstrations. We collect robot demonstrations using a joystick to teleoperate the robot. At each timestep, we collect two images of size 256×256 from the robot and table mounted cameras (I_{ego} , I_{env} respectively), the robot’s state x , and the robot action a applied by the human demonstrator. Robot state

x is a 7-dimensional vector including the end-effector pose and a boolean gripper state, the robot action is also a 7-dimensional vector consisting of linear and angular end-effector velocity along with the gripper actuation. To detect the segmentation masks of the objects we use the open-source langSAM model [47] built on SegmentAnything [48] and GroundingDINO [49]. langSAM allows us to extract bounding boxes $\{B\}$ for all the objects in the environment which are concatenated together to form the segmentation feature ϕ_{seg} . Notably, while langSAM provides bounding boxes for everything present in the environment, the robot still doesn't explicitly know which bounding box is important for task completion.

Results. Figure 6 summarizes the results of our real-world experiments. As in simulated experiments, we consider two robot scenes: first, an In-Distribution (ID) scene where the observations are from the same distribution as the training set, and second, an Out-of-Distribution (OOD) scene in which a distributional shift is introduced by changing the color of the table. We placed a white sheet on the surface — note that this also removed reflections and changed the intensity of the shadows. These ID and OOD scenes are shown in the left column of Figure 6. Each method is evaluated over 20 rollouts per scene, and we report the success rates averaged over three random seeds.

Across all three tasks, TransMASK consistently outperforms all the baselines in the ID scene. Our method significantly outperforms BC and VAE in the OOD scene as well. It also substantially outperforms VINN in both scenes on *Pick* and *Stack*. However, in *Scoop*, TransMASK achieves a substantial gain in performance in ID and only a modest improvement in OOD. Similarly, compared to CLASS our method achieves slight improvement in the success rate in OOD. Notably, our method is trained exclusively on demonstrations collected in the ID scene, **whereas VINN and CLASS are trained on a mixed dataset containing equal numbers of ID and OOD demonstrations**. Although performance in the ID scene is generally higher than in the OOD scene, the degradation under distribution shift is less pronounced than in the simulated experiments. This is primarily explained by the fact that the target object positions vary less drastically in the real-world setup — it is sampled from a smaller region, with variations within 5 cm.

VI. CONCLUSION

In this manuscript we explored the problem of robust imitation learning under environmental variations. One way to make learned policies more robust is by pruning the states they are conditioned on; instead of a policy which depends on every observed feature, we want the robot to only reason over features that are relevant to the current task. To this end, we introduced TransMASK, a modular mask which can be added to a variety of standard imitation learning frameworks. TransMASK does not require modifying the imitation learning objective or the training procedure; instead, it relies on the intrinsic gradients of the optimization scheme. Specifically, TransMASK learns a sparse mask matrix that transforms the input state into a compressed representation by eliminating

irrelevant components (e.g., background color, table texture, clutter). The resulting representation preserves features that significantly influence the policy's loss, thereby aligning the learned representations with task structure. By conditioning the policy on this compressed state representation, TransMASK produces policies that generalize better to distributional shifts.

Additionally, we analyzed the commonly used representation strategies in imitation learning, including the information bottleneck objective and contrastive learning. We showed how these approaches can inadvertently introduce instability or eliminate task-critical information due to the misalignment of the auxiliary objective with policy optimization. By contrast, TransMASK directly exploits gradients from the primary imitation objective, ensuring that the feature selection is tightly coupled to task performance. The structure of M also prevents TransMASK from learning a state embedding which collapses into an action representation.

We tested TransMASK through experiments with state-of-the-art baselines in both simulated environments and real-world robotic manipulation tasks. Across all settings, particularly under environmental perturbation, TransMASK consistently achieved significantly higher success rates, demonstrating improved robustness.

Limitations and Future Work. Our core limitation lies in our assumption that the input space admits a clear separation between task-relevant and task-irrelevant features, i.e., the state s is sufficiently disentangled into μ and η . In practice, this disentanglement is not guaranteed. Object-level segmentation offers a promising mechanism to approximate this structure by explicitly decomposing the scene into semantically meaningful features. Although segmentation models introduce their own failure modes, our real-world experimental results indicate that these do not constitute a significant bottleneck for our proposed method. A second limitation arises from the implicit learning of the mask. The mask is not trained with direct supervision; instead, it is optimized through policy gradients during training. Consequently, its quality depends on the stability of the optimization process and the quality of the training data. In cases where the loss landscape is noisy or training data is limited, the policy may converge to a suboptimal solution, leading the mask to encode incorrect features. This issue is particularly likely in small datasets, where irrelevant features may exhibit accidental correlations with actions, causing the mask to include irrelevant features in the state representation. Therefore, our method should be applied with careful consideration of data quality, dataset size, and optimization stability.

While our experiments focus on imitation learning, the underlying principle is not restricted to this setting. The approach could potentially be extended to reinforcement learning, particularly in scenarios involving sim-to-real transfer, where robustness to distribution shift and the extraction of task-relevant features are critical. Future work will focus on exploring the application of TransMASK to this application. Another potential avenue for future research is to provide theoretical guarantee for the convergence of the mask.

REFERENCES

- [1] Josh Tobin et al. Domain randomization for transferring deep neural networks from simulation to the real world. In *International Conference on Intelligent Robots and Systems*, 2017.
- [2] Oleg Kaidanov et al. The role of domain randomization in training diffusion policies for whole-body humanoid control. *arXiv preprint arXiv:2411.01349*, 2024.
- [3] Mohit Sharma, Claudio Fantacci, Yuxiang Zhou, Skanda Koppula, Nicolas Heess, Jon Scholz, and Yusuf Aytaç. Lossless adaptation of pretrained vision models for robotic manipulation. *arXiv preprint arXiv:2304.06600*, 2023.
- [4] Jingyun Yang, Max Sobol Mark, Brandon Vu, Archit Sharma, Jeannette Bohg, and Chelsea Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *International Conference on Robotics and Automation*, 2024.
- [5] Rouhollah Rahmatizadeh, Pooya Abolghasemi, Ladislau Bölöni, and Sergey Levine. Vision-based multi-task manipulation for inexpensive robots using end-to-end learning from demonstration. In *International Conference on Robotics and Automation*, 2018.
- [6] Jinghuan Shang and Michael S Ryoo. Self-supervised disentangled representation learning for third-person imitation learning. In *International Conference on Intelligent Robots and Systems*, 2021.
- [7] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [8] Sung-Wook Lee, Xuhui Kang, Brandon Y Yang, and Yen-Ling Kuo. Class: Contrastive learning via action sequence supervision for robot manipulation. In *Conference on Robot Learning*, 2025.
- [9] Vittorio Giammarino, James Queeney, and Ioannis Ch Paschalidis. Visually robust adversarial imitation learning from videos with contrastive learning. In *International Conference on Robotics and Automation*, 2025.
- [10] Cheng Chi et al. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2025.
- [11] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, and Krzysztof Choromanski. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.
- [12] Pranay Gupta, Henny Admoni, and Andrea Bajcsy. Adapting by analogy: Ood generalization of visuomotor policies via functional correspondence. *arXiv preprint arXiv:2506.12678*, 2025.
- [13] Jilei Mao et al. Omnid: Generalizable robot manipulation policy via image-based bev representation. *arXiv preprint arXiv:2508.11898*, 2025.
- [14] Vaibhav Saxena et al. What matters in learning from large-scale datasets for robot manipulation. *arXiv preprint arXiv:2506.13536*, 2025.
- [15] Modi Shi et al. Is diversity all you need for scalable robotic manipulation? *arXiv preprint arXiv:2507.06219*, 2025.
- [16] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, and Abhishek Gupta. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration. In *International Conference on Robotics and Automation*, 2024.
- [17] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, 2021.
- [18] Sagar Parekh, Heramb Nemlekar, and Dylan P Losey. Towards balanced behavior cloning from imbalanced datasets. *Autonomous Robots*, 2026.
- [19] Joey Hejna et al. Re-mix: Optimizing data mixtures for large scale imitation learning. *arXiv preprint arXiv:2408.14037*, 2024.
- [20] Haoran Xu, Xianyu Zhan, Honglei Yin, and Huiling Qin. Discriminator-weighted offline imitation learning from suboptimal demonstrations. In *International Conference on Machine Learning*, 2022.
- [21] Geon-Hyeong Kim et al. Demodice: Offline imitation learning with supplementary imperfect demonstrations. In *International Conference on Learning Representations*, 2021.
- [22] Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. *arXiv preprint arXiv:2310.12972*, 2023.
- [23] Jung Yeon Park and Lawson Wong. Robust imitation of a few demonstrations with a backwards model. *Advances in Neural Information Processing Systems*, 2022.
- [24] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Advances in Neural Information Processing Systems*, 2016.
- [25] Xin Zhang, Yanhua Li, Ziming Zhang, and Zhi-Li Zhang. f-gail: Learning f-divergence for generative adversarial imitation learning. *Advances in Neural Information Processing Systems*, 2020.
- [26] Jianren Wang, Ziwen Zhuang, Yuyang Wang, and Hang Zhao. Adversarially robust imitation learning. In *Conference on Robot Learning*, 2022.
- [27] Dahuin Jung, Hyungyu Lee, and Sungroh Yoon. Sample-efficient adversarial imitation learning. *Journal of Machine Learning Research*, 2024.
- [28] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. Vima: Robot manipulation with multimodal prompts. 2023.
- [29] Andy Zeng, Pete Florence, Jonathan Tompson, Stefan Welker, Jonathan Chien, Maria Attarian, Travis Armstrong, Ivan Krasin, Dan Duong, Vikas Sindhwani, et al. Transporter networks: Rearranging the visual world for robotic manipulation. In *Conference on Robot Learning*, 2021.
- [30] Jonathan Yang, Chelsea Finn, and Dorsa Sadigh. Invariance co-training for robot visual generalization. *arXiv preprint arXiv:2512.05230*, 2025.
- [31] Jean-Bastien Grill et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in Neural Information Processing Systems*, 2020.
- [32] Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. *arXiv preprint arXiv:2112.01511*, 2021.
- [33] Bian Xi Han, Oscar Mendez, and Simon Hadfield. Skill-il: Disentangling skill and knowledge in multitask imitation learning. In *International Conference on Intelligent Robots and Systems*, 2022.
- [34] Michael Noseworthy, Rohan Paul, Subho Roy, Daehyung Park, and Nicholas Roy. Task-conditioned variational autoencoders for learning movement primitives. In *Conference on Robot Learning*, 2020.
- [35] Jongjin Park, Younggyo Seo, Chang Liu, Li Zhao, Tao Qin, Jinwoo Shin, and Tie-Yan Liu. Object-aware regularization for addressing causal confusion in imitation learning. *Advances in Neural Information Processing Systems*, 2021.
- [36] Alessandro Achille and Stefano Soatto. Emergence of invariance and disentanglement in deep representations. *Journal of Machine Learning Research*, 2018.
- [37] Alessandro Achille and Stefano Soatto. A separation principle for control in the age of deep learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 2018.
- [38] Shuanghao Bai et al. Rethinking latent redundancy in behavior cloning: An information bottleneck approach for robot manipulation. *arXiv preprint arXiv:2502.02853*, 2025.
- [39] Ritwik Bera, Vinicius G Goecks, Gregory M Gremillion, Vernon J Lawhern, John Valasek, and Nicholas R Waytowich. Gaze-informed multi-objective imitation learning from human demonstrations. *arXiv preprint arXiv:2102.13008*, 2021.
- [40] Robert M Gray. *Entropy and information theory*. Springer Science & Business Media, 2011.
- [41] Ashvin Nair, Shikhar Bahl, Alexander Khazatsky, Vitchyr Pong, Glen Berseth, and Sergey Levine. Contextual imagined goals for self-supervised robotic learning. In *Conference on Robot Learning*, 2020.
- [42] Ashish Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [43] Mariusz Bojarski et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- [44] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [45] Quentin Gallouédec, Nicolas Cazin, Emmanuel Dellandréa, and Liming Chen. panda-gym: Open-source goal-conditioned environments for robotic learning. *arXiv preprint arXiv:2106.13687*, 2021.
- [46] Hans-Peter Piepho. An algorithm for a letter-based representation of all-pairwise comparisons. *Journal of Computational and Graphical Statistics*, 2004.
- [47] Luca Medeiros. Lang segment anything: Segment anything model with text prompts. <https://github.com/luca-medeiros/lang-segment-anything.git>, 2025. GitHub repository, accessed January 14, 2025.
- [48] Nikhila Ravi et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- [49] Shilong Liu et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.